# Implementation of Traffic Analysis and Use of Data Analytics with a Machine Learning Approach

**Alena Krause, Shannon Soulard, Shelly Davidashvilly**

College of Engineering and Computer Science
Florida Atlantic University
akrause2017@fau.edu, ssoulard2018@fau.edu, sdavidashvilly2018@fau.edu

2 December 2021

## 1 Abstract

With the ever-growing popularity of the internet, there arises a constant demand to create secure networks that are safe from attackers. As a result, many systems have started to use traffic analysis and data analytics to combat malicious activity. This use of traffic analysis helps to identify anomalies and potential security risks when applied to the field of cybersecurity. Detection of such malicious activity over a network is necessary, as it is the first step in creating a secure digital space. If a threat can be detected, then proper actions can be set in motion as a response.

The contributions of this project revolve around the following problem: How can traffic analysis and data analytics be used to categorize network traffic as either malicious or benign? This implementation aims to create a usable machine learning approach, in which artificial intelligence is used to classify network packets into one of the two aforementioned categories. Malicious packets describe harmful transactions occurring over a network. In the case of this implementation, the malicious attack type represents a DoS, or Denial-of-Service attack, where adversaries work to block legitimate requests to a system by reducing its availability. In contrast, benign packets describe normal traffic occurring over a network.

Three distinct machine learning architectures were constructed: including an ANN, DNN, and a custom "proposed" model. Each of the architectures were trained on the existing UNSW-NB15 dataset and performed exceptionally well with high accuracy. The same model architectures were later applied to a custom dataset. This dataset was created by simulating a DoS attack and capturing traffic using an open-source packet analyzer tool. The models again performed with very high accuracy.

The successfully implemented solution to the proposed problem showed that a machine learning model is able to classify traffic data, in the form of packets, as either harmful or normal. Such models have the potential to monitor traffic over a network to determine when a DoS attack is occurring and could feasibly alert network administrators of suspicious activity.

## 2 Introduction

The research team proposes a new approach for the implementation of machine learning models that are able to correctly identify abnormalities when presented with raw network traffic. The following sections detail the research process, and discuss motivations and contributions, preliminary background materials, the implementation procedure and results, and finally some concluding remarks.

### 2.1 Motivations and Contributions

The motivation for this research stemmed from the desire to study and analyze how traffic analysis is used in cybersecurity. The acquired knowledge was applied to the creation of the aforementioned custom monitoring

system. The existing literature on the subject makes use of established datasets containing large amounts of network traffic, used to train highly complex neural networks. The literature, however, is sparse when it comes to machine learning approaches that use more realistic and/or real-time data. As a result, this paper aims to explore machine learning classification abilities when presented with more practical values.

This implementation focused on the replication of existing models, found in literature, that were trained on a simplified version of an established dataset. This was done to create a baseline for comparison against the performance of the model architectures when using a custom dataset. By creating a custom dataset, conclusions could then be drawn about how the model architectures performed on more realistic and easily capturable data. If a system is able to correctly classify network traffic from within a complex dataset as well as from within a more realistic one, it has good potential for being applied to networks in the real world.

# 3 Preliminaries

To promote understanding of the topics discussed, it is necessary to provide a brief overview of some of the concepts and terminology used within the scope of this paper. Network traffic analysis (NTA) is a method for monitoring the activity occurring within a network. It can be used to identify performance issues while also uncovering any present security threats. Administrators can utilize traffic analysis to discover information about a network, including the traffic flowing through it and the source of the data [1]. A short list of applications for traffic analysis is described below:

- Detecting unknown threats.
- Detecting malware communications over a network.
- Detecting suspiciously high traffic rates.

Traffic analysis methodologies can be used to collect data on Transmission Control Protocol (TCP) and Internet Protocol (IP) style packets that are being routed in a network.

A packet refers to a unit of data that is routed between some source and destination over the internet. Packets contain control information as well as user data. The control information provides instructions for delivering the payload. Because packets hold valuable information, they are an attractive target for attackers that wish to manipulate their contents or perhaps replicate them. In a Denial-of-Service (DoS) attack, the flooding of packets is utilized to make multiple requests over a network with the intent of rendering it unresponsive. Sending multiple requests via packets overloads the system, as there is excessive traffic. This usually will cause legitimate requests to be denied over the network. The type of attack that this implementation attempted to simulate, within an isolated environment, was a SYN flood attack. TCP SYN flood (SYN flood) is a type of DoS attack that disrupts the TCP routing sequence to overwork resources on a server, causing it to become unresponsive.

## 3.1 Machine Learning Concepts

Machine Learning (ML) is a branch of artificial intelligence, where implemented algorithms can improve automatically with repeated training on given data. Because machine learning improves with the repetition of data exposure, the technique can be utilized for traffic and data analysis. The following list contains the various evaluation metrics that were used to determine how well the implemented models performed:

- **Accuracy**: The fraction of predictions the model got right.
- **Precision**: The number of true positives (correctly predicted positive values) divided by the total number of positive predictions.
- **Recall**: The number of true positives that were found.
- **F1**: The weighted average of precision and recall.
- **AUC**: The measure of the ability of a classifier to distinguish between classes (area under the curve).
- **Loss**: The prediction error of a neural network.

# 4 Literature Review

The literature on traffic analysis utilizes a variety of supervised and unsupervised machine learning techniques, as well as non-machine learning based methods. A wide range of attack types and publicly available datasets are commonly considered.

## 4.1 Supervised Learning

Supervised learning techniques implement models that learn a mathematical function to map input to output via labeled data. When applied to traffic analysis and anomaly detection, the input data consists of various characterizations of normal traffic and network attacks, labeled accordingly. The algorithms can be used to predict the labels of new data in intrusion detection systems. [29] utilized Decision Trees, Ripple Rule, Random Forest, and Bayesian Networks for an intrusion detection system. The approach was broken into small components that served to detect if an attack was made. The "Pre-Processor Part" takes care of data preprocessing, with the output being fed into a machine learning sub-component. Eventually the data is routed to the "Protection Part", where an IP address can be blocked if an attack was detected. The authors of [6] developed an intrusion detection system, for use in healthcare, to protect against common attacks such as DoS, and utilized a Support Vector Machine (SVM) model. In [28], the authors proposed a hybrid model, based on J48 and Bayesian Network techniques. It enabled classification and feature selection when using the SCADA dataset. [7] tested four supervised machine learning techniques on the UNSW-NB15 dataset - Support Vector Machines, Random Forests, Decision Trees, and Naive Bayes Classifiers. Reference [11] utilized the Random Forest method on the KDDcup99 dataset, whereas [25] employed a decision tree on an older version of the dataset. [4] showed that Support Vector Machines and Random Forests were useful in detecting network traffic anomalies in business networks. [17] tested Decision Trees, J48, Random Forests, and the best performing REP tree model, on the publicly available SNMP-MIB dataset. Deep learning methods were tested by [10] and showed that deep neural networks (as well as autoencoders) were efficient. They also demonstrated that the first three octets of IP addresses were useful in handling dynamic IPs. [15] employed recurrent neural networks, both in their general form and in their long-short-term-memory form (LSTM). [3] utilized the ISOT-CID dataset and added a novel feature designed to reduce the size of packet flow, consequently supporting classification. The implemented process balanced the dataset and tested Decision Tree, Neural Networks, K-Nearest Neighbors (KNN), Naive Bayes, Support Vector Machines (SVM), and Random Forest classifiers with cross and split validation. The results showed that Decision Trees and Random Forests had the best performance, whereas Naive Bayes and Support Vectors Machines had the worst.

## 4.2 Unsupervised Learning

Unsupervised learning works to discover properties in data without the use of pre-defined labels. They are popularly employed to discover groups or other relevant patterns in data. Alternatively, they can be used as a preprocessing step for supervised learning in order to provide more meaningful data. The authors in [22], for instance, employed a Restricted Boltzmann machine-based intrusion detection system for wireless sensor networks.

Clustering techniques are popular unsupervised machine learning methods that, when provided unlabeled data, learn how to best group or categorize based on inherent properties. For the case of anomaly detection in traffic analysis, implemented models learn how to distinguish normal traffic from attacks based on the input features provided. References [13] and [14], evaluated the use of k-means clustering for this kind of detection. [23] utilized autoencoders and sparse autoencoders, as part of a proposed autonomous and self-adaptive misuse detection system. Their implementation achieved 77.99% accuracy.

## 4.3 Non-Machine Learning Based Approaches

Non-machine learning based methods typically employ code that analyzes network packets. Using preset conditions, these programs determine if presented traffic is benign or malicious. For instance, the authors in [5] analyzed Apache Web Server's access-log file in order to detect SQL injections and XSS attacks. They justified the use of log files, describing them as easily available since they require no expensive hardware for analysis. A mentioned con of log files remarked that they do not include all data of an HTTP request or response. A python script was implemented to analyze the log files and worked to check for an SQLI or XSS attack. [24] employed a similar tactic, but instead analyzed network packets rather than log files, claiming they can contain more useful information. The implementation started with data preprocessing and checked the type of the HTTP request at each data-point. It analyzed the packets, extracting key features and artifacts from the relevant fields, ultimately raising alerts within the system and logging any detection of attacks.

## 4.4 Literature Gaps

Although the literature successfully tests a variety of models on many publicly available datasets, there is less effort put into developing models and assessing their performances using collected real-time data. Additionally, the publicly available datasets that are commonly used often include features that are difficult or unrealistic to

capture. For instance, the UNSW-NB15 dataset consists of a mix of features that are easily capturable and features that are manually engineered, the latter of which can be harder to replicate and understand. Developing systems that use easily captured features, directly extracted from network packet information, can make the classification process more efficient and easy to understand. Therefore, the proposed approach makes an effort to omit features that are manually engineered and instead utilizes more simple classifiers.

# 5 Implementation Process

The project objective aimed to train three machine learning models on the existing UNSW-NB15 dataset. Later, the same model architectures were applied to a uniquely created, custom dataset.

## 5.1 Required Materials

This section details the various materials that were utilized for implementation. The machine learning models were constructed using Python and paired with libraries such as Numpy for scientific computations, Pandas for the data analysis and preprocessing, Matplot for displaying graphs, Keras to create the neural networks, and Tensorflow for training and inferences related to the neural networks. The data capturing software used to aid in the creation of the custom dataset was the open-source packet capturing tool, Wireshark. The materials required for the simulation and simultaneous capturing of DoS attack data included: a disconnected router with several paired devices, the Bash command language, and a Linux based CLI ping tool - hping3.

## 5.2 Model Architectures

An artificial neural network (ANN) and a deep neural network (DNN), mirrored from literature [1], were two models used for classification. Additionally, a more simple - proposed model was created. The artificial neural network consisted of one hidden layer using activation ReLu and contained 850 nodes. The output layer used the softmax activation. It used the Adam optimizer in conjunction with the binary cross entropy loss function, paired with a learning rate of 0.001. The model was set to train for 100 epochs with a batch size of 100. The deep neural network contained three hidden layers, each with 100 nodes and activation ReLu. It utilized the same output layer configuration along with the same optimizer, loss function, learning rate, epoch value, and batch size. The proposed custom model contained two hidden layers, each with 4 nodes and activation ReLu. It additionally utilized the dropout technique with a rate of 40%. The optimizer, loss function, and output layer all featured the same configurations as the previously described architectures. The epoch value and batch size were both reduced to 50.

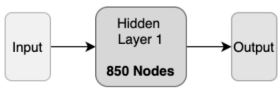
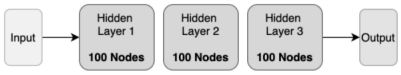
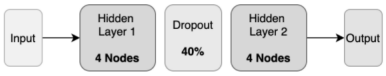
| ANN | DNN | Custom/Proposed |
|---|---|---|
|  |  |  |
| Epochs: 100<br>Batch size: 100<br>Optimizer: Adam (*LR = 0.001)<br>Loss: *Binary cross entropy<br>Hidden layer activation: ReLu<br>Output layer activation: Softmax | Epochs: 100<br>Batch size: 100<br>Optimizer: Adam (*LR = 0.001)<br>Loss: *Binary cross entropy<br>Hidden layer activation: ReLu<br>Output layer activation: Softmax | Epochs: 50<br>Batch size: 50<br>Optimizer: Adam (LR = 0.001)<br>Loss: Binary cross entropy<br>Hidden layer activation: ReLu<br>Output layer activation: Softmax |

**Figure 1:** *Model Architectures*

## 5.3 UNSW-NB15 Dataset

The UNSW-NB15 dataset was created by researchers at the University of South Wales in Australia. It includes over 100GB of raw network traffic, with 49 features. The classification categories within this dataset include nine different attack types: fuzzers, analysis, backdoors, DoS, exploits, generic, reconnaissance, shellcode, and worms (note that the dataset also contains samples of normal traffic as well)[19]. For this implementation, the dataset was reduced to only include DoS and normal data.

In order to process the data within the UNSW-NB15 dataset, the following techniques were applied. All of the data points for the benign traffic within the dataset were originally categorized as NaN (not a number), therefore it was imperative that these labels were changed to have a value of 0. This label change would allow for the removal of any of the true missing data by searching for rows containing NaN. As previously mentioned, all of the data points that did not have an attack category of either DoS or normal were removed from the dataset. For the sake of comparability and usability, it was concluded that it would be best to only use features that could easily be captured using Wireshark. Consequently, several of the UNSW-NB15 features were removed. Only the features that could feasibly translate to Wireshark data were kept. Additionally, duplicated values within the dataset were removed. After this preprocessing, the resulting data was extremely unbalanced. To fix this, the extraction of a smaller subset of data points was necessary in order to balance the categories. Before balancing, over 99% of the data points were benign. After balancing this number reduced to 72%, see Figure 2.

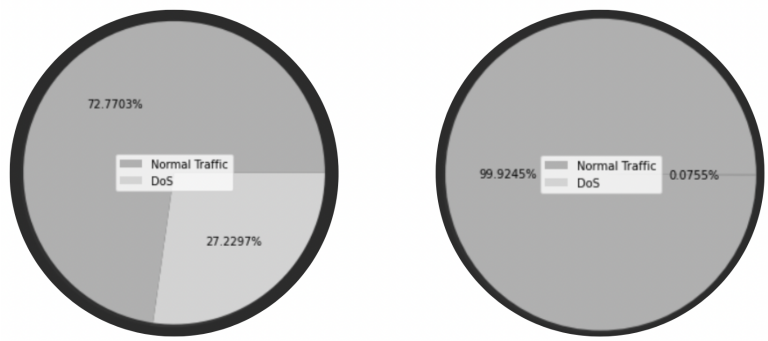



**Figure 2:** *UNSW-NB15 Data Balancing*

To further prepare the data for the model, all of the categorical columns were one-hot encoded. The balanced data was separated from their labels into Numpy arrays. From here, the data was split into train, validation, and test sets. Finally, normalization was applied using the MinMaxScaler technique.

The features utilized in the implementation are detailed in Figure 3, with corresponding descriptions derived from the UNSW-NB15 documentation [19].

## 5.4 Dataset Creation

To create the custom dataset, it was imperative that an isolated environment was set up in order to safely simulate a denial-of-service attack. To do this, multiple devices were connected to an access point that was disconnected from the internet. Wireshark enabled the viewing of packets being sent within a given network. The tool was used to capture normal TCP packets over public WiFi. It was also used to capture the simulated DoS attack data from within the isolated environment.

## 5.5 Features

This section discusses the features captured using Wireshark. Time describes the timestamp of the packet. Source IP and port denote the address where the packet is coming from. Similarly, the destination IP and port show where the packet is going. The protocol is the specific internet convention used to transfer the packet. The length is the count of bytes pertaining to a particular packet. Time to Live (TTL), references the amount of time a router keeps a packet within a network, before discarding. The sequence number counts every byte that is sent out by a specific host. The acknowledgement number, similarly, counts every byte that has been received. The next sequence number is calculated using the sum of the last-seen sequence number and the segment length. Window size notes the available buffer space designated for receiving packets. Finally, flags give instructions on how to handle an individual TCP packet. For the machine learning model, both time and IP values were omitted, as those mainly serve as identification features and would not be useful.

| | |
|---|---|
| sport | Source port number |
| dsport | Destination port number |
| proto | Transaction protocol |
| state | Indicates to the state and its dependent protocol |
| dur | Record total duration |
| sbytes | Source to destination transaction bytes |
| dbytes | Destination to source transaction bytes |
| sttl | source to destination time to live value |
| dttl | Destination to source time to live value |
| sloss | Source packets retransmitted or dropped |
| dloss | Destination packets retransmitted or dropped |
| service | http, ftp, smtp, ssh, dns, ftp-data ,irc and (-) if not much used service |
| Sload | Source bits per second |
| Dload | Destination bits per second |
| Spkts | Source to destination packet count |
| Dpkts | Destination to source packet count |
| swin | Source TCP window advertisement value |
| dwin | Destination TCP window advertisement value |
| stcpb | Source TCP base sequence number |
| dtcpb | Destination TCP base sequence number |
| attack_cat | The name of each attack category. In this data set , nine categories e.g. Fuzzers, Analysis, Backdoors, DoS Exploits, Generic, Reconnaissance, Shellcode and Worms |
| Label | 0 for normal and 1 for attack records |

**Figure 3:** *UNSW-NB15 Features*

## 5.6 Simulation

Within the isolated environment, the router was used for the connection of two devices. One device was used to capture traffic data via Wireshark, while the other was used to send hundreds of thousands of packets over the network. Note that the router's built in DoS protection and other security features were disabled by signing into the router's UI settings panel.

To randomize the data as much as possible, five different bash scripts were created. Each bash script used an open source packet generator to flood 10,000 packets to a designated IP address. Random values were injected for the packet lengths, source addresses, and time to live values. Within each of the scripts, one out of five possible destination ports was specified. These five ports were the most common ports being used in the captured normal data. To run the bash commands, fifteen terminal windows within the same device were used. Enabling the execution of each of the scripts three times. An example bash script command is shown as follows:

```
sudo hping3 -S -c 10000
-d $(( ($RANDOM % 1000) + 100))
-t $(( ($RANDOM % 200) + 50))
-p 63049 --flood
--rand-source [DESTINATION IP ADDRESS]
```

The above command performs the following actions. The -S specifier indicates the type of packets to send, SYN packets. -c describes the number of packets to send (10000). The -d command is used to indicate the size of the packets; here, a random size is injected using bash. Time-to-live is specified using -t, also randomized using bash. -p is the destination port, in this case it was specified as one out of five different values, based on frequent occurrences within the normal dataset. Finally, –rand-source generates a spoofed IP address to act as the packet's source IP. At the end of the command, the destination IP address is included.

Fifteen scripts were used because randomizing every packet greatly slowed down the rate of packet sending, to the point where it was no longer a DoS attack. It was necessary to send as many random packets as possible without slowing down the rate of sending. Randomized data would allow for the machine learning model to make more meaningful connections about the data in order to classify it. Figure 4 shows a sample table of captured packet data with corresponding features.

| | No. | Time | Source Port | Destination Port | Protocol | Length | Time to Live | Sequence Number (raw) | Calculated window size | Total Length | Flags | Next Sequence Number | Frame Number | Acknowledgment number (raw) | Label |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0.000000 | 56069 | 443 | TLSv1.2 | 271 | 128 | 944798163 | 514 | 257 | 0x40 | 218 | 1 | 2723284736 | 0.0 |
| **1** | 2 | 0.032784 | 443 | 56069 | TLSv1.2 | 249 | 227 | 2723284736 | 23 | 235 | 0x40 | 196 | 2 | 944798380 | 0.0 |
| **2** | 3 | 0.087226 | 56069 | 443 | TCP | 54 | 128 | 944798380 | 514 | 40 | 0x40 | 218 | 3 | 2723284931 | 0.0 |
| **3** | 4 | 2.382655 | 62971 | 443 | TLSv1.2 | 137 | 128 | 104831187 | 512 | 123 | 0x40 | 84 | 4 | 1015133480 | 0.0 |
| **4** | 5 | 2.405408 | 443 | 62971 | TCP | 60 | 56 | 1015133480 | 270 | 40 | 0x00 | 1 | 5 | 104831270 | 0.0 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **2943** | 2944 | 764610.658696 | 43330 | 54676 | TCP | 527 | 193 | 2090893118 | 512 | 513 | 0x00 | 474 | 2944 | 218234718 | 1.0 |
| **2944** | 2945 | 764610.658696 | 43331 | 54676 | TCP | 527 | 193 | 218544318 | 512 | 513 | 0x00 | 474 | 2945 | 877316256 | 1.0 |
| **2945** | 2946 | 764610.658696 | 43332 | 54676 | TCP | 527 | 193 | 1752956111 | 512 | 513 | 0x00 | 474 | 2946 | 605204384 | 1.0 |
| **2946** | 2947 | 764610.658696 | 60982 | 54682 | TCP | 1070 | 139 | 1870939604 | 512 | 1056 | 0x00 | 1017 | 2947 | 1426365054 | 1.0 |
| **2947** | 2948 | 764610.658696 | 60983 | 54682 | TCP | 1070 | 139 | 618191558 | 512 | 1056 | 0x00 | 1017 | 2948 | 419631120 | 1.0 |

**Figure 4:** *Captured Data Sample*

## 5.7 Simulation Output

When the attacking device was connected directly to the router via Ethernet, the receiving device experienced an average reception speed of 300Mbps. When connected wirelessly to the router, this number reduced to 200 Mbps. Overall, the receiving device experienced severe slowdowns specifically when using Wireshark.

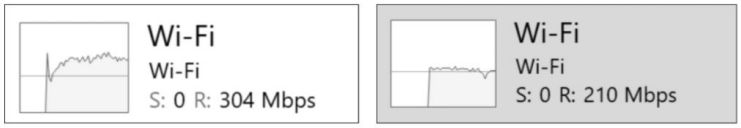



**Figure 5:** *Simulation Impact*

The final capture results from the simulation resulted in millions of packet dissections. In order to closely match the baseline dataset, extraction of a smaller subset of the data was needed. This ultimately resulted in a dataset that consisted of 69.4% normal traffic and 30.5% DoS. Note that these numbers closely match the percentages used for the balanced version of the UNSW-NB15 dataset, thus enabling meaningful comparisons and conclusions to be drawn about the data captured and the models that were constructed. The same data preprocessing techniques were applied to the captured data.

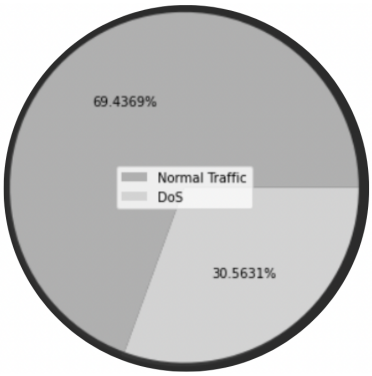



**Figure 6:** *Captured Data Balancing*

# 6 Implementation Results

The custom and literature models were evaluated based on their ability to correctly classify elements within the UNSW-NB15 dataset as well as within the constructed dataset. The metrics considered for performance evaluation included: accuracy, precision, recall, F1 score, and AUC. The number of parameters each model used for training

was also considered and is noted in the graphs below. This gives an indication of the complexity of the model architectures.

All models, regardless of the dataset, were able to achieve very high, often perfect metrics. This was presumed to be a result of the data simplification that occurred when reducing many features and only considering DoS attacks. The original UNSW-NB15 dataset included 49 features, whereas the custom analysis selected only 20. There were originally 9 attack types (Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode and Worms). DoS was the only attack type kept for analysis, since it could be feasibly replicated for the purposes of the experiment.

The custom model was designed to have significantly less parameters and applied more generalization techniques than the referenced literature models. This was done in order to compare a simple architecture against more complex models. In terms of raw numbers for both the UNSW-NB15 dataset and the custom dataset, the proposed model either met or came very close to the performances of the parameter heavy, deeper models referenced from the literature. Figure 7 shows the accuracy comparisons of all the models on the UNSW-NB15 dataset. Figure 8 shows the performances of the models on the simulated data.

All models achieved perfect accuracy on the UNSW-NB15 data, with slightly higher results than what the papers reported (note that these papers used the full dataset). For the simulated data, the custom model had slightly lower performance, with an average score of 99.78 across all metrics. The other models, comparatively achieved a score of 100.

Figures 9 and 10 show the accuracy and loss curves of all the models when exposed to the UNSW-NB15 dataset and simulated data, respectively. The curves provide additional insight into the training process of the models. The more complex models showed little to no overfitting between the training and validation sets and had stable loss and accuracy curves. The custom model displayed more overfitting and better performance on the validation set when compared to the training set. This can be attributed to the validation set being less representative of the data, implying that its use made it easier for the simple model to make predictions. Despite these observations, the custom model met or came very close to the performance scores of the complex networks. This could signify that the literature models were unnecessarily complex, while the proposed model was perhaps too simple for the task at hand. Perhaps a model with medium complexity could meet the same performance with less parameters; feasibly executing with minimal overfitting and without sacrificing model training stability.

| | Params: 142,802 | Params: 37,002 | Params: 694 |
|---|---|---|---|
| | ANN | DNN | Custom |
| Accuracy | 100 | 100 | 100 |
| Precision | 100 | 100 | 100 |
| Recall | 100 | 100 | 100 |
| F1 | 100 | 100 | 100 |
| AUC | 100 | 100 | 100 |
| | *Paper reported* ***99.26%*** *accuracy with full dataset* | *Paper reported* ***99.22%*** *accuracy with full dataset* | |

**Figure 7:** *UNSW-NB15 Results*

| | Params: 17,852 | Params: 22,302 | Params: 106 |
|---|---|---|---|
| | ANN | DNN | Custom |
| Accuracy | 100 | 100 | 99.77 |
| Precision | 100 | 100 | 99.77 |
| Recall | 100 | 100 | 99.76 |
| F1 | 100 | 100 | 99.77 |
| AUC | 100 | 100 | 99.83 |

**Figure 8:** *Captured Data Results*

# 7 Concluding Remarks

Considering only DoS attacks and normal traffic, when paired with the reduction of the feature space within the UNSW-NB15 dataset, caused great simplification of the information and allowed the simpler proposed model

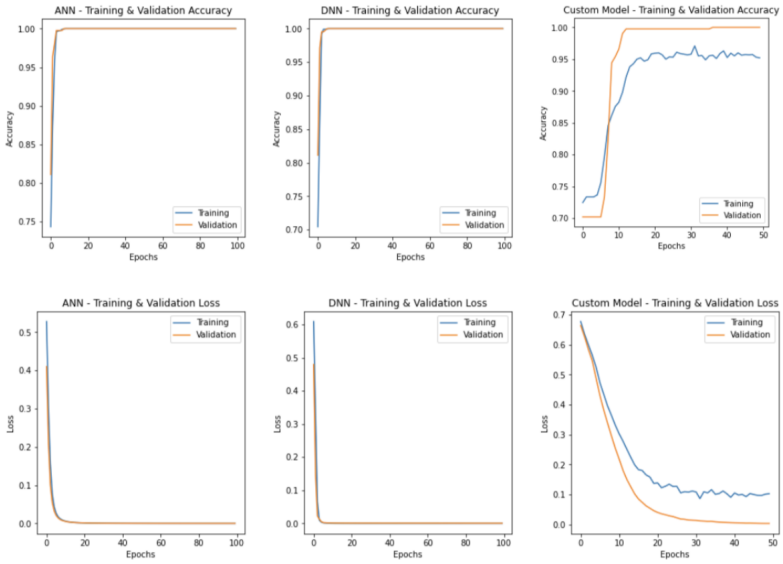



**Figure 9:** *UNSW-NB15 Accuracy and Loss*

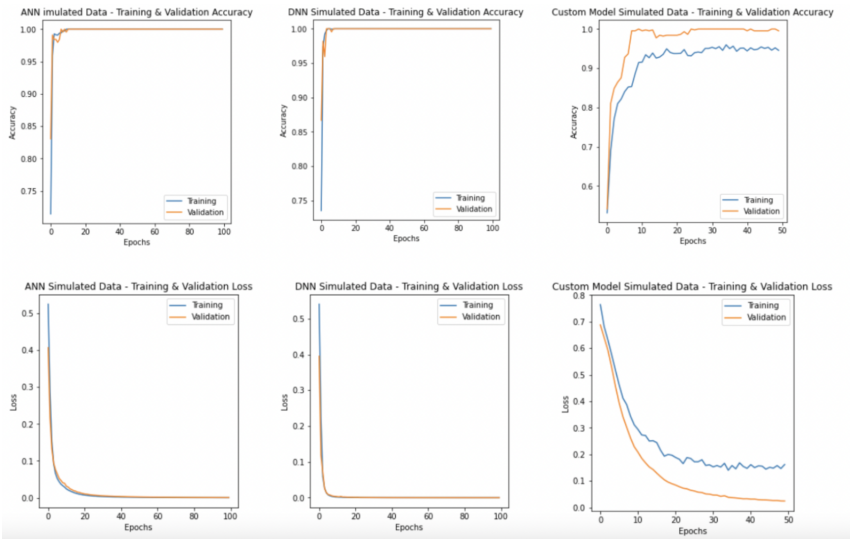



**Figure 10:** *Captured Data Accuracy and Loss*

to perform just as well as the deep models. The use of more shallow, simple models can prove to be beneficial. In some cases their implementation can be more efficient, namely, when used for real-time intrusion detection systems, as they can run faster. In terms of raw numerical values, the simple-proposed model achieved perfect scores on all metrics considered, when analyzing the UNSW-NB15 dataset, as did the literature models. The proposed custom model had slightly worse performance on the simulated data, fluctuating near values of 99.80%. The loss and accuracy curves additionally shed light on the greater overfitting and instability that occurred when using the simple model. Although, in the end, the resulting classifications were just as good.

These results indicate that simple models can be successful. However, extra measures should still be taken to combat any potential overfitting and instability. Based on the performance of the models using the existing data set and the custom data set, it can be concluded that the architectures are efficient when presented with both complex and more simple/realistic features. That is, features that are more easily capturable over a network can still be used for intrusion detection systems.

Some of the main challenges that were faced included the randomization of packet data for the DoS simulation and overcoming data inconsistencies within the UNSW-NB15 dataset. Only fifteen types of random packets were created. Future exploration of this implementation would see that more random packets are generated to provide more complex data for the model to train on.

Simplifying the UNSW-NB15 dataset required the application of several data preprocessing techniques in order to obtain a use-able subset. The final subset was extremely small when compared to the original data. In

the future, it would be useful to evaluate the models on larger datasets. It would also be interesting to see how the model architectures perform when exposed to different attack types (i.e. not just DoS).

## References


[1] Ahmed A Mohammed Nan M Sahar Ahmed Aleesa Mohammed Younis Thanoun. "DEEP-INTRUSION DETECTION SYSTEM WITH ENHANCED UNSW-NB15 DATASET BASED ON DEEP LEARNING TECHNIQUES". In: *Journal of Engineering Science and Technology* (2021). URL: https://www.researchgate.net/publication/349670982_DEEP-INTRUSION_DETECTION_SYSTEM_WITH_ENHANCED_UNSW-NB15_DATASET_BASED_ON_DEEP_LEARNING_TECHNIQUES.

[2] Nour Alqudaha and Qussai Yaseen. "Machine Learning for Traffic Analysis: A Review". In: *Procedia Computer Science* 170 (Apr. 2020), pp. 911–916. URL: https://www.sciencedirect.com/science/article/pii/S1877050920305494.

[3] Amirah Alshammari and Abdulaziz Aldribi. "Apply machine learning techniques to detect malicious network traffic in cloud computing". In: *Journal of Big Data* 8 (June 2021). DOI: 10.1186/s40537-021-00475-1.

[4] Simon Duque Anton et al. "Evaluation of Machine Learning-based Anomaly Detection Algorithms on an Industrial Modbus/TCP Data Set". In: *Proceedings of the 13th International Conference on Availability, Reliability and Security* (2018). DOI: 10.1145/3230833.3232818. URL: http://dx.doi.org/10.1145/3230833.3232818.

[5] Merve Baş Seyyar, Ferhat Özgür Çatak, and Ensar Gül. "Detection of attack-targeted scans from the Apache HTTP Server access logs". In: *Applied Computing and Informatics* 14.1 (2018), pp. 28–36. ISSN: 2210-8327. DOI: https://doi.org/10.1016/j.aci.2017.04.002. URL: https://www.sciencedirect.com/science/article/pii/S2210832717300169.

[6] MohammadReza Begli, Farnaz Derakhshan, and Hadis Karimipour. "A Layered Intrusion Detection System for Critical Infrastructure Using Machine Learning". In: Aug. 2019, pp. 120–124. DOI: 10.1109/SEGE.2019.8859950.

[7] Mustapha Belouch, Salah El hadaj, and Mohamed Idhammad. "Performance evaluation of intrusion detection based on machine learning using Apache Spark". In: *Procedia Computer Science* 127 (Jan. 2018), pp. 1–6. DOI: 10.1016/j.procs.2018.01.091.

[8] Glenn Carl et al. "Denial-of-Service Attack Detection Techniques". In: *IEEE Computer Science* (Jan. 2006), pp. 82–89. URL: https://ieeexplore.ieee.org/document/1580418.

[9] Leslie F.Sikos. "Packet analysis for network forensics: A comprehensive survey". In: *Forensic Science International: Digital Investigation* 32 (Mar. 2020). URL: https://www.sciencedirect.com/science/article/pii/S1742287619302002.

[10] Sahil Garg et al. "A Hybrid Deep Learning-Based Model for Anomaly Detection in Cloud Datacenter Networks". In: *IEEE Transactions on Network and Service Management* 16.3 (2019), pp. 924–935. DOI: 10.1109/TNSM.2019.2927886.

[11] Rajiv Gurjwar and Bhura Parul. "A Review on attacks classification using decision tree algorithms". In: (Apr. 2019).

[12] "HOW TO PERFORM TCP SYN FLOOD DOS ATTACK DETECT IT WITH WIRESHARK". In: *Network Protocol Analyzers* (). URL: https://www.firewall.cx/general-topics-reviews/network-protocol-analyzers/1224-performing-tcp-syn-flood-attack-and-detecting-it-with-wireshark.html.

[13] Meng Jianliang, Shang Haikun, and Bian Ling. "The Application on Intrusion Detection Based on K-means Cluster Algorithm". In: *2009 International Forum on Information Technology and Applications*. Vol. 1. 2009, pp. 150–152. DOI: 10.1109/IFITA.2009.34.

[14] R. Kumari et al. "Anomaly detection in network traffic using K-mean clustering". In: *2016 3rd International Conference on Recent Advances in Information Technology (RAIT)*. 2016, pp. 387–393. DOI: 10.1109/RAIT.2016.7507933.

[15] Donghwoon Kwon et al. "A survey of deep learning-based network anomaly detection". In: *Cluster Computing* 22 (2017), pp. 949–961.

[16] Batta Mahesh. "Machine Learning Algorithms - A Review". In: *International Journal of Science and Research (IJSR)* 9 Issue 1.12 (Jan. 2001), pp. 1–6. URL: https://www.researchgate.net/profile/Batta-Mahesh/publication/344717762_Machine_Learning_Algorithms_-A_Review/links/5f8b2365299bf1b53e2d243a/Machine-Learning-Algorithms-A-Review.pdf.

[17] Abdelrahman Manna and Mouhamad Alkasassbeh. "Detecting network anomalies using machine learning and SNMP-MIB dataset with IP group". In: *2019 2nd International Conference on new Trends in Computing Sciences (ICTCS)*. 2019, pp. 1–5. DOI: 10.1109/ICTCS.2019.8923043.

[18] Nour Moustafa, Gideon Creech, and Jill Slay. "Big Data Analytics for Intrusion Detection System: Statistical Decision-Making Using Finite Dirichlet Mixture Models". In: *Data Analytics and Decision Support for Cybersecurity: Trends, Methodologies and Applications*. Ed. by Iván Palomares Carrascosa, Harsha Kumara Kalutarage, and Yan Huang. Cham: Springer International Publishing, 2017, pp. 127–156. ISBN: 978-3-319-59439-2. DOI: 10.1007/978-3-319-59439-2_5. URL: https://doi.org/10.1007/978-3-319-59439-2_5.

[19] Nour Moustafa and Jill Slay. "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)". In: *2015 Military Communications and Information Systems Conference (MilCIS)*. 2015, pp. 1–6. DOI: 10.1109/MilCIS.2015.7348942.

[20] Nour Moustafa and Jill Slay. "The evaluation of Network Anomaly Detection Systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set". In: *Information Security Journal: A Global Perspective* 25.1-3 (2016), pp. 18–31. DOI: 10.1080/19393555.2015.1125974. eprint: https://doi.org/10.1080/19393555.2015.1125974. URL: https://doi.org/10.1080/19393555.2015.1125974.

[21] Nour Moustafa, Jill Slay, and Gideon Creech. "Novel Geometric Area Analysis Technique for Anomaly Detection Using Trapezoidal Area Estimation on Large-Scale Networks". In: *IEEE Transactions on Big Data* 5.4 (2019), pp. 481–494. DOI: 10.1109/TBDATA.2017.2715166.

[22] Safa Otoum, Burak Kantarci, and Hussein T. Mouftah. "On the Feasibility of Deep Learning in Sensor Network Intrusion Detection". In: *IEEE Networking Letters* 1.2 (2019), pp. 68–71. DOI: 10.1109/LNET.2019.2901792.

[23] Dimitrios Papamartzivanos, Félix Gómez Mármol, and Georgios Kambourakis. "Introducing Deep Learning Self-Adaptive Misuse Network Intrusion Detection Systems". In: *IEEE Access* 7 (2019), pp. 13546–13560. DOI: 10.1109/ACCESS.2019.2893871.

[24] Mansi Patel, S Raja Prabhu, and Animesh Kumar Agrawal. "Network Traffic Analysis for Real-Time Detection of Cyber Attacks". In: *2021 8th International Conference on Computing for Sustainable Global Development (INDIACom)*. 2021, pp. 642–646. DOI: 10.1109/INDIACom51348.2021.00113.

[25] Kai Peng et al. "Intrusion Detection System Based on Decision Tree over Big Data in Fog Environment". In: *Wireless Communications and Mobile Computing* 2018 (Mar. 2018). DOI: 10.1155/2018/4680867.

[26] Mohanad Sarhan et al. "NetFlow Datasets for Machine Learning-Based Network Intrusion Detection Systems". In: *Big Data Technologies and Applications* (2021), 117–135. ISSN: 1867-822X. DOI: 10.1007/978-3-030-72802-1_9. URL: http://dx.doi.org/10.1007/978-3-030-72802-1_9.

[27] Christoph L. Schuba et al. "Analysis of a Denial of Service Attack on TCP". In: *? ?? ()*, pp. 208–223. URL: https://www.researchgate.net/publication/2793223_Analysis_of_a_Denial_of_Service_Attack_on_TCP.

[28] Imtiaz Ullah and Qusay H. Mahmoud. "A hybrid model for anomaly-based intrusion detection in SCADA networks". In: *2017 IEEE International Conference on Big Data (Big Data)* (2017), pp. 2160–2167.

[29] N. Wattanapongsakorn et al. "A Practical Network-Based Intrusion Detection and Prevention System". In: *2012 IEEE 11th International Conference on Trust, Security and Privacy in Computing and Communications*. 2012, pp. 209–214. DOI: 10.1109/TrustCom.2012.46.